SESSION 7 — Array Manipulation

Array manipulation functions are essential when:

- Combining datasets
- Splitting data for training/testing
- Modifying array structure
- Adding or removing elements
- Managing memory correctly

---

1. np.concatenate()

Why It Exists

To join arrays along an existing axis.

Syntax

np.concatenate((arr1, arr2), axis=0)

Parameters

- tuple of arrays → arrays to combine
- axis → 0 (rows), 1 (columns)

Important

Arrays must have same shape except along concatenation axis.

```
import numpy as np

a = np.array([[1,2],[3,4]])
b = np.array([[5,6],[7,8]])
c = np.array([[9,10],[11,12]])

print("Concatenate axis=0:\n", np.concatenate((a,b,c)))
print("Concatenate axis=0:\n", np.concatenate((a,b,c), axis=0))
print("Concatenate axis=1:\n", np.concatenate((a,b,c), axis=1))
```

```
Concatenate axis=0:
 [[ 1  2]
 [ 3  4]
 [ 5  6]
 [ 7  8]
 [ 9 10]
 [11 12]]
Concatenate axis=0:
 [[ 1  2]
 [ 3  4]
 [ 5  6]
 [ 7  8]
 [ 9 10]
 [11 12]]
Concatenate axis=1:
 [[ 1  2  5  6  9 10]
 [ 3  4  7  8 11 12]]
```

2. Stacking Functions

Stacking adds a new dimension or stacks arrays in specific orientation.

np.vstack()

Vertical stacking (row-wise)

np.hstack()

Horizontal stacking (column-wise)

np.dstack()

Depth stacking (third dimension)

```
print("vstack:\n", np.vstack((a,b,c)))
print("hstack:\n", np.hstack((a,b,c)))
print("dstack:\n", np.dstack((a,b,c)))
print("dstack:\n", np.dstack((a,b,)))
```

```
vstack:
 [[ 1  2]
 [ 3  4]
 [ 5  6]
 [ 7  8]
 [ 9 10]
 [11 12]]
hstack:
 [[ 1  2  5  6  9 10]
 [ 3  4  7  8 11 12]]
dstack:
 [[[ 1  5  9]
  [ 2  6 10]]

 [[ 3  7 11]
  [ 4  8 12]]]
dstack:
 [[[1 5]
  [2 6]]

 [[3 7]
  [4 8]]]
```

Stacking vs Concatenation

Concatenation:

- Joins along an existing axis.

Stacking:

- May introduce new axis (especially dstack).

Use stacking when creating higher dimensional arrays.

3. Splitting Arrays

np.split()

Splits array into equal parts.

Syntax: np.split(arr, sections, axis=0)

np.array_split()

Allows unequal splitting.

Why two functions?

np.split() requires exact division. np.array_split() handles uneven splits.

```
arr = np.arange(8)
print(arr)
print("Split:", np.split(arr, 4))
print("Array Split:", np.array_split(arr, 3))
```

```
[0 1 2 3 4 5 6 7]
Split: [array([0, 1]), array([2, 3]), array([4, 5]), array([6, 7])]
Array Split: [array([0, 1, 2]), array([3, 4, 5]), array([6, 7])]
```

4. Insert, Append, Delete

np.insert(arr, index, values, axis=None)

Inserts values before given index.

np.append(arr, values, axis=None)

Appends values at end.

np.delete(arr, index, axis=None)

Deletes elements at index.

Important: These return new arrays (original not modified).

```
arr = np.array([10,20,30])

print("Insert:", np.insert(arr, 1, 15))
print(arr)
print("Append:", np.append(arr, 40))
print(arr)
print("Delete:", np.delete(arr, 1))
print(arr)
```

```
Insert: [10 15 20 30]
[10 20 30]
Append: [10 20 30 40]
[10 20 30]
Delete: [10 30]
[10 20 30]
```

5. Copy vs View

Understanding memory behavior is critical.

View (.view())

Shares memory with original array.

Copy (.copy())

Creates independent memory.

Modifying view affects original. Modifying copy does not.

```
arr = np.array([1,2,3])

view_arr = arr.view()
copy_arr = arr.copy()

arr[0] = 100

print("Original:", arr)
print("View:", view_arr)
print("Copy:", copy_arr)
```

```
Original: [100   2   3]
View: [100   2   3]
Copy: [1 2 3]
```

Q1.

Create an array from 1 to 15.
Split it into 5 equal parts.

Q2.

Create an array from 1 to 11.
Split it into 4 parts using `array_split()`.

Q3.

Given: arr = np.array([10, 20, 30, 40, 50])

Insert 25 at index 2.

Q4.

Given: arr = np.array([[1,2], [3,4]])

Insert a new row [5,6] at index 1.

Q5.

Given: arr = np.array([5, 10, 15])

Append 20 and 25 at the end.

Q6.

Given: arr = np.array([[1,2], [3,4]])

Append column [7,8] to this array.

Q7.

Given: arr = np.array([100, 200, 300, 400, 500])

Delete: a) Element at index 3
b) First two elements

Q8.

Given: arr = np.array([[1,2], [3,4], [5,6]])

Delete the second row.

Q9.

Create: arr = np.array([1,2,3,4])

Create:

- view_arr using view()
- copy_arr using copy()

Modify arr[0] = 999

Print all three arrays. Explain what happened.

Q10.

Create array from 1 to 6. Reshape it into 2×3 matrix. Change element at position (0,1) to 100.

Q11.

Given: arr = np.array([[1,2,3], [4,5,6]])

Apply:

- flatten()
- ravel()

Modify first element of both results. Observe difference.

Q12.

Create array from 1 to 12. Split into 3 equal parts. Delete last element from each part. Combine all remaining elements into a single array.

1.

Create a 4×4 matrix using np.arange(1,17).

- Extract the second row.
- Extract the last column.
- Compute the mean of the entire matrix.

2.

Create array: arr = [5, 10, 15, 20, 25, 30]

- Extract elements greater than 10 AND less than 30.
- Find their cumulative sum.

3.

Generate 10 random integers between 1 and 50.

- Sort them.
- Return indices that would sort the original array.
- Find the 75th percentile.

4.

Create two 2×3 matrices.

- Concatenate them row-wise.
- Then split the result into 2 equal parts.

5.

Create array: arr = [1, 2, 2, 3, 4, 4, 5]

- Remove duplicate values.
- Reverse the array using slicing.
- Compute standard deviation.

6.

Create a 5×5 matrix using np.arange(1,26).

- Extract all elements greater than 10 and less than 20.
- Replace those values with 0 using boolean indexing.
- Compute total sum of modified matrix.

7.

Generate 1000 random numbers from normal distribution.

- Compute mean and standard deviation.
- Clip values between -1 and 1.
- Count how many values were clipped.

8.

Create array: arr = [10, 20, np.nan, 40, 50, np.nan]

- Compute mean ignoring NaN.
- Replace NaN values with 0.
- Compute new mean.

9.

Create two matrices: A (3×3) using np.arange() B (3×3) random integers between 1 and 10.

- Perform element-wise multiplication.
- Compute 90th percentile of result.

- Transpose the result.

10.

Create array: arr = np.arange(1,21)

- Reshape into 4×5 matrix.
- Extract elements divisible by both 2 and 3.
- Insert value 999 at index 2.
- Delete last element.
- Verify whether all remaining values are positive.

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```